Name: Dazon, Joshua Aareon Danielle P.	**Date Performed: 2/12/2026**
Course/Section: CPE31S1	**Date Submitted: 2/12/2026**
Instructor: Engr. Robin Valenzuela	**Semester and SY: 2nd Sem 2025-2026**

Activity 6: Targeting Specific Nodes and Managing Services

1. **Objectives:**

1.1 Individualize hosts
1.2 Apply tags in selecting plays to run
1.3 Managing Services from remote servers using playbooks
1.4 Managing Ansible Roles for Modularization

2. **Intended Learning Outcomes:**

- Differentiate and organize remote hosts into appropriate inventory groups based on their server roles (e.g., web, database, file servers).
- Analyze playbook execution results and interpret changes made across different remote servers.
- Configure and manage system services remotely using Ansible modules (e.g., service module for starting and enabling services).
- Design and apply Ansible Roles to modularize configurations and improve playbook reusability, scalability, and maintainability.

3. **Discussion**:

In this activity, we try to individualize hosts. For example, we don't want apache on all our servers, or maybe only one of our servers is a web server, or maybe we have different servers like database or file servers running different things on different categories of servers and that is what we are going to take a look at in this activity.

We also try to manage services that do not automatically run using the automations in playbook. For example, when we install web servers or httpd for CentOS, we notice that the service did not start automatically.

Requirement:

In this activity, you will need to create another Ubuntu VM and name it Server 3. Likewise, you need to activate the second adapter to a host-only adapter after the installations. Take note of the IP address of the Server 3. Make sure to use the command *ssh-copy-id* to copy the public key to Server 3. Verify if you can successfully SSH to Server 3.

Task 1: Targeting Specific Nodes

1. Create a new playbook and named it site.yml. Follow the commands as shown in the image below. Make sure to save the file and exit.

```
---

- hosts: all
  become: true
  tasks:

  - name: install apache and php for Ubuntu servers
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

   - name: install apache and php for CentOS servers
     dnf:
       name:
         - httpd
         - php
       state: latest
     when: ansible_distribution == "CentOS"
```

2. Edit the inventory file. Remove the variables we put in our last activity and group according to the image shown below:

```
[web_servers]
192.168.56.120
192.168.56.121

[db_servers]
192.168.56.122


[file_servers]
192.168.56.123
```

   Make sure to save the file and exit.

   Right now, we have created groups in our inventory file and put each server in its own group. In other cases, you can have a server be a member of multiple groups, for example you have a test server that is also a web server.

3. Edit the *site.yml* by following the image below:

```
---

- hosts: all
  become: true
  pre_tasks:

  - name: install updates (CentOS)
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install updates (Ubuntu)
    apt:
      upgrade: dist
      update_cache: yes
    when: ansible_distribution == "Ubuntu"


- hosts: web_servers
  become: true
  tasks:

  - name: install apache and php for Ubuntu servers
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: install apache and php for CentOS servers
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

```
HOA_6 > ! site.yml
---
- hosts: web_servers
  become: true
  pre_tasks:

  - name: install updates (CentOS)
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "CentOS"


  - name: install updates (Ubuntu)
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "Ubuntu"



- hosts: web_servers
  become: true
  tasks:

  - name: Install apache and php for Ubuntu Servers
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
```

The *pre-tasks* command tells the ansible to run it before any other thing. In the *pre-tasks*, CentOS will install updates while Ubuntu will upgrade its distribution package. This will run before running the second play, which is targeted at *web_servers*. In the second play, apache and php will be installed on both Ubuntu servers and CentOS servers.

Run the *site.yml* file and describe the result.

4. Let's try to edit again the *site.yml* file. This time, we are going to add plays targeting the other servers. This time we target the *db_servers* by adding it on the current *site.yml*. Below is an example: (Note add this at the end of the playbooks from task 1.3.

```
- hosts: db_servers
  become: true
  tasks:

  - name: install mariadb package (CentOS)
    yum:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb- Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true

  - name: install mariadb packege (Ubuntu)
    apt:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "Ubuntu"
```

Make sure to save the file and exit.

```
HOA_6 > ! site.yml
- hosts: web_servers

  - name: install apache and php for CentOS Servers
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"

- hosts: db_servers
  become: true
  tasks:

  - name: Install mariaDB package (CentOS)
    yum:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb - Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true

  - name: Install mariadb package (Ubuntu)
    apt:
      name: mariadb-server

Debug Console (Ctrl+Shift+Y)
    when: ansible_distribution == "Ubuntu"
```

Run the *site.yml* file and describe the result.

5. Go to the remote server (Ubuntu) terminal that belongs to the db_servers group and check the status for mariadb installation using the command: *systemctl status mariadb.* Do this on the CentOS server also.

```
joshua-dazon@Server1:~$ systemctl status mariadb
● mariadb.service - MariaDB 10.11.13 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; pres
     Active: active (running) since Thu 2026-02-12 16:39:41 PST; 1h 38min a
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
    Process: 1279 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d
    Process: 1315 ExecStartPre=/bin/sh -c systemctl unset-environment _WSRE
    Process: 1322 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ]
    Process: 1498 ExecStartPost=/bin/sh -c systemctl unset-environment _WSR
    Process: 1520 ExecStartPost=/etc/mysql/debian-start (code=exited, statu
   Main PID: 1450 (mariadbd)
     Status: "Taking your SQL requests now..."
      Tasks: 10 (limit: 30367)
     Memory: 126.8M (peak: 130.0M)
        CPU: 1.661s
     CGroup: /system.slice/mariadb.service
             └─1450 /usr/sbin/mariadbd

Feb 12 16:39:41 Server1 mariadbd[1450]: 2026-02-12 16:39:41 0 [Note] Plugin
Feb 12 16:39:41 Server1 mariadbd[1450]: 2026-02-12 16:39:41 0 [Note] InnoDB
Feb 12 16:39:41 Server1 mariadbd[1450]: 2026-02-12 16:39:41 0 [Warning] You
```

Describe the output.

- The mariadb is already installed and activated

6. Edit the *site.yml* again. This time we will append the code to configure installation on the *file_servers* group. We can add the following on our file.

```
- hosts: file_servers
  become: true
  tasks:

  - name: install samba package
    package:
      name: samba
      state: latest
```

Make sure to save the file and exit.

Run the *site.yml* file and describe the result.

The testing of the *file_servers* is beyond the scope of this activity, and as well as our topics and objectives. However, in this activity we were able to show that we can target hosts or servers using grouping in ansible playbooks.

Task 2: Using Tags in running playbooks

In this task, our goal is to add metadata to our plays so that we can only run the plays that we want to run, and not all the plays in our playbook.

1. Edit the *site.yml* file. Add tags to the playbook. After the name, we can place the tags: *name_of_tag*. This is an arbitrary command, which means you can use any name for a tag.

```
---

- hosts: all
  become: true
  pre_tasks:

  - name: install updates (CentOS)
    tags: always
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install updates (Ubuntu)
    tags: always
    apt:
      upgrade: dist
      update_cache: yes
    when: ansible_distribution == "Ubuntu"
```

```
- hosts: web_servers
  become: true
  tasks:

  - name: install apache and php for Ubuntu servers
    tags: apache,apache2,ubuntu
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: install apache and php for CentOS servers
    tags: apache,centos,httpd
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"
```

```
- hosts: db_servers
  become: true
  tasks:

  - name: install mariadb package (CentOS)
    tags: centos, db,mariadb
    dnf:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb- Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true

  - name: install mariadb packege (Ubuntu)
    tags: db, mariadb,ubuntu
    apt:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "Ubuntu"

- hosts: file_servers
  become: true
  tasks:

  - name: install samba package
    tags: samba
    package:
      name: samba
      state: latest
```

Make sure to save the file and exit.
Run the *site.yml* file and describe the result.

2. On the local machine, try to issue the following commands and describe each result:
   2.1 *ansible-playbook --list-tags site.yml*
   2.2 *ansible-playbook --tags centos --ask-become-pass site.yml*
   2.3 *ansible-playbook --tags db --ask-become-pass site.yml*
   2.4 *ansible-playbook --tags apache --ask-become-pass site.yml*
   2.5 *ansible-playbook --tags “apache,db” --ask-become-pass site.yml*

```
joshua-dazon@LocalMachine:~/CPE212_31S1/HOA_6$ ansible-playbook --list-tags site.yml -K

playbook: site.yml

  play #1 (all): all    TAGS: []
      TASK TAGS: [always]

  play #2 (web_servers): web_servers    TAGS: []
      TASK TAGS: [apache, centos, httpd]

  play #3 (db_servers): db_servers      TAGS: [mariadb,centos,db]
      TASK TAGS: [centos, db, mariadb, ubuntu]

  play #4 (file_servers): file_servers  TAGS: []
      TASK TAGS: [samba]
```

Task 3: Managing Services

1. Edit the file site.yml and add a play that will automatically start the httpd on CentOS server.

```
- name: install apache and php for CentOS servers
  tags: apache,centos,httpd
  dnf:
    name:
      - httpd
      - php
    state: latest
  when: ansible_distribution == "CentOS"

- name: start httpd (CentOS)
  tags: apache, centos,httpd
  service:
    name: httpd
    state: started
  when: ansible_distribution == "CentOS"
```

Figure 3.1.1

Make sure to save the file and exit.

You would also notice from our previous activity that we already created a module that runs a service.

```
- hosts: db_servers
  become: true
  tasks:

  - name: install mariadb package (CentOS)
    tags: centos, db,mariadb
    dnf:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb- Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true
```

Figure 3.1.2

This is because in CentOS, installed packages' services are not run automatically. Thus, we need to create the module to run it automatically.

2. To test it, before you run the saved playbook, go to the CentOS server and stop the currently running httpd using the command *sudo systemctl stop httpd.* When prompted, enter the sudo password. After that, open the browser and enter the CentOS server's IP address. You should not be getting a display because we stopped the httpd service already.
3. Go to the local machine and this time, run the *site.yml* file. Then after running the file, go again to the CentOS server and enter its IP address on the browser. Describe the result.
   To automatically enable the service every time we run the playbook, use the command *enabled: true* similar to Figure 7.1.2 and save the playbook.

Supplementary Activity:

1. **Creating a Role for Web Servers**
2. **Creating a Role for Web Servers**

HOA_6 > Web-Servers > tasks > ! main.yml

```yaml
---
# tasks file for Web-Servers


- name: install updates (Ubuntu)
  tags: always
  apt:
    upgrade: dist
    update_cache: yes
  when: ansible_distribution == "Ubuntu"



- name: Install apache and php for Ubuntu Servers
  apt:
    name:
      - apache2
      - libapache2-mod-php
    state: latest
  when: ansible_distribution == "Ubuntu"

- name: install apache and php for CentOS Servers
  tags: apache,centos,httpd
  dnf:
    name:
      - httpd
      - php
    state: latest
  when: ansible_distribution == "CentOS"
```

3. **Creating a Role for Database Servers**

```yaml
---
# tasks file for Database

  name: Install mariaDB package (CentOS)
  yum:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "CentOS"

  name: "Mariadb - Restarting/Enabling"
  service:
   name: mariadb
   state: restarted
  enabled: true

  name: Install mariadb package (Ubuntu)
  tags: db, mariadb, ubuntu
  apt:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "Ubuntu"
```

4. **Combining Multiple Roles in a Playbook**

To orchestrate multiple roles in a single playbook for different server groups. Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for database.

Reflections:

Answer the following:

1. What is the importance of putting our remote servers into groups?
   - It is easier to manage large infrastructures. Instead of targeting individual hosts, you can apply configurations to entire groups.
2. What is the importance of tags in playbooks?
   - Tags allow you to run only specific parts of a playbook instead of executing everything. This is crucial for large playbooks where you may only want to update one service or configuration.

3. Why do think some services need to be managed automatically in playbooks?

   - To ensure services are always running in the desired state